

1. What you built

Your main feature: a Java purchasing planner. It accepts required parts and supplier offers, then recommends which suppliers to use and how much to buy from each. It explains the decision and reports any shortage.

The business problem

A buyer needs parts before a deadline. Suppliers differ in price, delivery time, minimum order quantity and capacity. Bulk discounts mean the cheapest-looking supplier is not always the cheapest complete solution.

Your 60-second introduction

“I built a Java procurement backend that recommends supplier allocations for a demand list. It checks delivery deadlines, minimum quantities, capacity and bulk discounts. I use dynamic programming to maximize on-time fulfillment first and then minimize cost. The response includes allocations, costs, arrival dates and rejection reasons. I also added JWT security and supporting supplier and order APIs. The planner uses submitted snapshots and does not automatically place orders.”

What is complete, and what is not

The core planner is implemented. Ten focused automated tests passed from a fresh local clone using existing dependencies. A live gateway request returned the expected 900.00 plan. This is not proof of a completely clean-machine startup or a full integration test of every service. Docker and a frontend were deliberately skipped.

How to use this guide

Read pages 2-5 first. Explain the 900 versus 960 example without looking. Then read security, testing and limitations. Practice the questions aloud and locate the matching code. Use the sample answers only after you understand them.

2. Know the two different flows

Flow A: purchasing recommendation - the assignment

Client + bearer token → gateway → order-service → PurchasePlanner → JSON plan

POST /api/orders/purchase-plans receives demands and offers together. It does not fetch offers from MySQL or supplier-service. It does not save a plan, reserve stock, or create an order. Supplier codes in this request do not need existing supplier records.

Flow B: manual order creation - supporting feature

Client → gateway → order-service → Feign supplier lookup → supplier-service → order database

POST /api/orders receives a supplierId, item name, quantity and unit price chosen by the caller. The service verifies the supplier, then saves a CREATED order. A separate cancellation endpoint changes its status to CANCELLED.

Service / port	Responsibility
order-service / 8082	Planner plus persisted manual orders in order_db.
supplier-service / 8081	Supplier CRUD in supplier_db.
api-gateway / 8080	Entry point, routes and token validation.
auth-service / 8083	Two local accounts, JWT issuance and public keys.
discovery-server / 8761	Eureka registry for service discovery.
config-server / 8888	Loads central configuration from GitHub config-repo.

The planner is a pure Java class. Its algorithm can be tested without starting these services. The deployed order-service still needs its normal application dependencies at startup.

3. Understand the input and rules

Input	Meaning
planningDate	Date from which delivery lead time is measured.
Demand	partCode, quantity and neededBy deadline.
Offer	supplierCode, partCode, leadDays, minimumQuantity, capacity and tiers.
MOQ	Minimum order quantity: choose zero, or at least this amount.
Capacity	Maximum quantity available from this supplier for this part in this snapshot.
Tier	fromQuantity activates a unit price for the entire lot.

The objective has an order

First: supply as much demand as possible on time, without overbuying. **Second:** among plans supplying that quantity, choose the lowest purchase cost. A cheap plan supplying fewer units loses to a plan supplying more units.

All-units discount

If the base price is 12.00 and the tier from 80 units is 9.00, buying 100 costs $100 \times 9 = 900$. It is not 79×12 plus 21×9 . That second interpretation would be an incremental discount and is not implemented.

Validation and assumptions

One demand per part; 1-500 units per demand; at most 20 parts and 20 offers per part. Codes must be valid and duplicate supplier/part offers are rejected. Prices must be positive, fit cents exactly and not increase at higher tiers. Every offer needs a base tier starting at 1.

Parts are independent, quantities are integers, split sourcing is allowed, and overbuying is disabled. Arrival on the deadline is valid. Lead times use calendar days. Tax, freight, holidays and shared supplier-wide capacity are outside this model.

4. Explain the worked example

Demand: 100 BOLT-M8 units. Plan on 2026-09-08; delivery required by 2026-09-15.

Supplier	Constraints	Price
SUP-A	Lead 2 days; MOQ 20; capacity 60	10.00 base; 8.00 from 50
SUP-B	Lead 3 days; MOQ 10; capacity 100	12.00 base; 9.00 from 80
SUP-LATE	Lead 10 days; MOQ 1; capacity 100	1.00 base

Why SUP-LATE loses

It arrives on September 18, after the September 15 deadline. Its low price cannot make it eligible.

Why a greedy choice loses

Buying 60 from A activates 8.00, costing 480. The remaining 40 from B cost 12.00 each, or 480. Total = **960.00**. Buying all 100 from B activates 9.00 and costs **900.00**. The complete allocation is cheaper even though A has a lower bulk unit price.

Expected full response

FULFILLED; supplied 100; shortage 0; SUP-B quantity 100; arrival September 11; totalCost 900.00. The response explains the selected lot and why alternatives were unused or rejected.

Shortage variant

Change B capacity to 30. A can supply 60 and B can supply 30; the late offer remains invalid. The expected maximum is 90, leaving shortage 10. Cost = $60 \times 8 + 30 \times 12 = \mathbf{840.00}$. SHORTAGE is a valid plan response with HTTP 200, not a system failure.

5. Dynamic programming, simply

Dynamic programming remembers the cheapest way to reach each exact quantity, instead of repeatedly solving the same smaller problem.

State and initialization

previous[q] means the minimum cost of buying exactly q units from suppliers already processed. **previous[0] = 0**. Other quantities start as unreachable, represented by INF. Costs are long integer cents.

Process one supplier at a time

Start next as a copy of previous: that represents buying zero from this supplier. Try each legal lot x from MOQ to min(capacity, demand). For every total q at least x, compare the old best with **previous[q-x] + cost(x)**. Keep a strictly cheaper result.

Recurrence: $\text{next}[q] = \min(\text{next}[q], \text{previous}[q-x] + x \times \text{tierPrice}(x))$. All updates read the previous supplier row. Reading the new row could accidentally reuse the same supplier and exceed capacity.

Recover the actual recommendation

chosen[supplier][quantity] records the chosen lot when a state improves. After all suppliers, use the full-demand state if reachable; otherwise find the largest reachable quantity. Walk the chosen rows backwards, subtracting each lot, to reconstruct allocations.

Why the answer is exact within the model

Every legal supplier choice is either zero or a quantity in the enumerated range. For each exact total, the algorithm keeps the cheapest combination seen so far. Because parts are independent, each part can be optimized separately and costs summed.

Complexity and ties

The main DP takes $O(S \times Q^2)$ per part and $O(S \times Q)$ memory for reconstruction. Tier-price lookup adds $O(S \times Q \times T)$, where T is at most 20. Q is capped at 500. Suppliers are sorted by code and equal-cost states are retained, making results repeatable. This is not a fairness rule.

6. Security and resilience

Login and token validation

The login controller checks configured account passwords using BCrypt. It issues a signed RS256 JWT valid for 900 seconds. The private RSA key signs; the published public key verifies. Signing is not encryption: JWT claims can be read.

Gateway and business services validate signature, issuer, audience and expiry. asit has procurement.read and procurement.write scopes. viewer has read scope only. A valid viewer token cannot POST a purchase plan.

Explain the status codes

Code	Meaning in your application
200	Successful read, login or calculated plan, including a shortage.
201	A manual order or supplier was created.
400	Invalid planner/business input.
401 / 403	Missing or invalid authentication / insufficient permission.
404 / 409	Resource missing / duplicate supplier code.
503	Manual order supplier verification unavailable.

Feign and circuit breaker

For manual order creation, a Feign interceptor forwards the caller JWT to supplier-service. A supplierLookup circuit breaker protects this remote verification. It does not wrap the pure purchasing algorithm.

CLOSED allows calls. After enough failures it becomes OPEN and rejects calls. After the wait, HALF_OPEN allows trial calls. Configuration: minimum 5 counted calls, 50% failures, 10-second wait, 2 trial calls. Missing suppliers are ignored by failure counting. Feign timeouts are 3 seconds to connect and 5 seconds to read.

Do not claim a full OAuth2 authorization server, token revocation, refresh tokens, production key rotation or stock reservation. Those features are not implemented.

7. Find the code and demonstrate it

File / class	What to explain
planning/PurchasePlanner.java	Records define input/output; validate checks input; solve performs DP; price picks a tier.
planning/PurchasePlanController.java	POST mapping, delegates to engine, converts invalid input to HTTP 400.
service/OrderService.java	Separate manual order flow and protected supplier verification.
auth/.../AuthController.java	Login credentials, scopes, JWT claims and public key response.
config/SecurityConfig.java	Which requests are public and which require read/write scopes.
PurchasePlannerTest.java	Eight engine tests, including brute-force comparisons.
PurchasePlanControllerTest.java	Two standalone API contract tests.

Five-minute interview demo

1. Explain the demand and supplier constraints before opening Postman.
2. Log in at `/api/auth/login` and use the fresh token.
3. POST `examples/purchase-plan.json` to port 8080 at `/api/orders/purchase-plans`.
4. Explain the 900.00 result and the late supplier rejection.
5. Lower B capacity to 30 and explain the expected shortage.
6. Open `solve()` and point to previous, next and chosen.
7. Show the focused test results and state their scope.

Test command

From order-service run:

```
./mvnw -Dtest=PurchasePlannerTest,PurchasePlanControllerTest test
```

The 10 tests cover discounts, splitting, MOQ, deadlines, shortages, invalid input, multiple parts and ties. One test compares 200 small deterministic random cases against exhaustive enumeration. This is stronger than checking just one example, but it is not proof for every possible deployment.

8. Questions you should be ready for

Why not always choose the cheapest unit price?

“The final lot changes the discount tier. A locally cheap choice can force an expensive remainder. I compare complete feasible allocations.”

Why use BigDecimal and integer cents?

“Money should not use binary floating-point comparisons. The API uses BigDecimal; the optimizer converts valid prices to exact long cents.”

Why not a database query for the recommendation?

“The decision involves combinations of quantities and discounts. SQL stores data, while the pure Java engine evaluates the constrained combinations.”

Does this guarantee the cheapest plan?

“For the maximum feasible supplied quantity, yes, within the stated input limits and assumptions. It excludes freight, shared capacities and other unmodeled constraints.”

Why six services for this assignment?

“The repository includes supporting service architecture, but it is more infrastructure than the core problem needs. A modular monolith would simplify deployment. The planner is isolated so its correctness does not depend on that architecture.”

How would you handle bigger demands or shared capacity?

“The current quantity-based DP would become expensive, and shared capacity couples parts. I would revisit the model and consider an integer optimization solver rather than claim this DP scales unchanged.”

How did AI help?

“I used AI assistance for implementation and documentation. I verified behavior through tests and demonstrations.” Only add that you can explain or modify every part once you have practiced it. The assignment explicitly allows AI but expects live understanding.

9. Your preparation plan

First 15 minutes: business understanding

Explain demand, MOQ, capacity, deadline and all-units discount in your own words. Calculate 960, 900 and 840 on paper. Describe why a shortage is not an HTTP error.

Next 20 minutes: trace the algorithm

Use demand 3 and two suppliers: A has MOQ 1, capacity 2, price 5; B has MOQ 1, capacity 3, price 4. After A: [0,5,10,INF]. After B: [0,4,8,12]. The answer is B3 for 12. These numbers are currency units for teaching; code stores cents.

Then change B capacity to 2: the best full plan is $A1 + B2 = 13$. Point to where previous-row reads prevent B being used twice. Practice reconstructing the chosen quantities.

Next 15 minutes: practice a live change

Change the sample deadline, one MOQ or a capacity. Predict the result first, run it, and explain any difference. Find the input limit check in `validate()`. Do not change authentication merely to bypass a failing demo.

Last 10 minutes: limitations and evidence

Be ready to say: no persisted plans, no capacity reservation, no automatic order conversion, no frontend or Docker, and no fresh-machine startup verification. Explain that these are explicit scope decisions, not hidden completed features.

Before the interview

Start MySQL, Config Server, Eureka, auth, supplier, order and gateway. Keep the README, example JSON, planner source and tests open. Get a new token. Never include passwords or tokens in presentation material.

You are ready when: you can explain the example, trace one DP transition, distinguish a plan from an order, explain 401 versus 403, and describe one limitation without reading this guide.

Source basis: local README, docs/PURCHASING-PLANNER.md, PurchasePlanner.java and current service configuration, reviewed 8 September 2026. This guide describes the implementation rather than promising production readiness.